Deep Learning Project Report - CIFAR-10 Image Classification

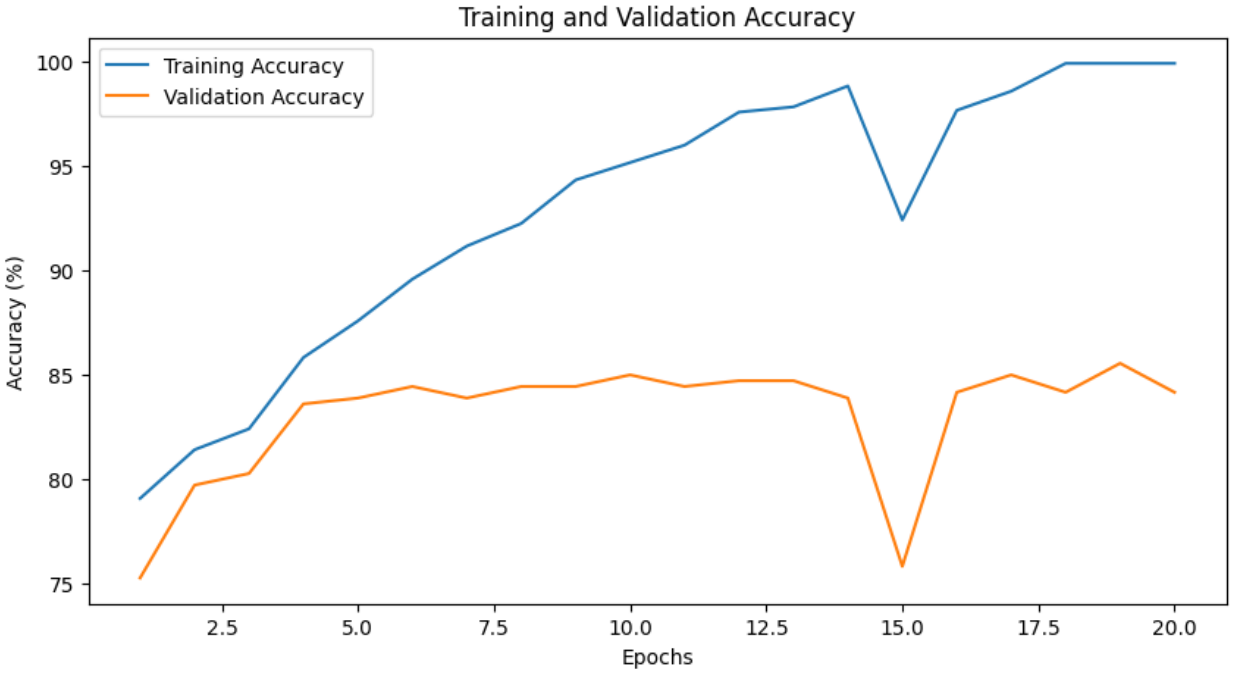
1. Introduction

This project was conducted to build and evaluate a Convolutional Neural Network (CNN) model for classifying images from the CIFAR-10 dataset using PyTorch. The model was trained to classify images into three categories: buildings, forests, and glaciers.

Model Training:

The current model demonstrates good performance (84.67% test accuracy) with clear room for improvement, particularly in its ability to generalize to new data. Future efforts should focus on reducing the gap between training and validation accuracy.

Graph Analysis:



The graph shows that the model achieved good performance (>85% validation accuracy) with potential for improvement in generalization. The gap between the training and validation curves indicates that improving generalization should be the primary focus for future development.

Q1: Learning Rate Comparison

0.001	High (89.17%)	More stable
0.01	Fluctuating (75-85%)	Less stable

The model achieved its best performance with a **Batch Size=32** and a **learning rate of 0.001** over 30 epochs, reaching a validation accuracy of **89.17%**. A gap between training and validation accuracy (overfitting) was observed, suggesting the need for generalization improvement techniques such as **Dropout** or **data augmentation**. **Early Stopping** with a learning rate of 0.001 requires further tuning of the **patience** parameter for better results.

Q2: Main Results (Dropout Configurations)

Configuration	Training Accuracy	Validation Accuracy	Test Accuracy
Config1 (Dropout=0.5)	91.08%	82.78%	85.11%
Config2 (Dropout=0.3)	90.75%	84.17%	85.56%
Config3 (No Dropout)	94.33%	85.00%	86.67%

The experiments showed that using **Dropout=0.3 (Config2)** achieved the best balance between performance and generalization, with a **test accuracy of 85.56%** and a reduced gap between training and validation. In contrast, the model without Dropout (Config3) achieved the highest accuracy (**86.67%**) but exhibited clear signs of **overfitting**. It is recommended to use **Dropout (0.3-0.5) in dense layers**, while avoiding excessive application in early convolutional layers.

Q3: Main Results (Filter Configurations)

Configuration	Training Accuracy	Validation Accuracy	Test Accuracy
Original (32,32,16)	94.83%	83.61%	85.56%

Config1 (16,16,16)	98.42%	84.17%	85.33%
Config2 (64,64,16)	93.00%	86.39%	87.11%

The results showed that **increasing the number of filters** in convolutional layers from 32 to 64 (**Config2**) improved test accuracy by **1.55%** compared to the original model, while reducing them to 16 (**Config1**) led to a slight performance drop. It is recommended to use **Config2** with regularization techniques (e.g., Dropout) to achieve the best balance between performance and generalization.

Q4: Main Results (Pooling Configurations)

Configuration	Training Accuracy	Validation Accuracy	Test Accuracy
Original (Avg-Max)	97.42%	85.28%	85.78%
Config1 (Max-Max)	90.92%	84.72%	85.56%
Config2 (Avg-Avg)	91.92%	83.61%	83.33%

The results showed that the **original model (Avg-Max)** achieved the **highest test accuracy (85.78%)**, while **Max-Max** scored **85.56%** and **Avg-Avg** scored **83.33%**. This confirms that **combining both pooling types** yields more balanced results, as **AvgPool preserves statistical information** in early layers, while **MaxPool focuses on the most important features** in later layers.

Q6: Main Results (Activation Functions)

Activation Function	Validation Accuracy
ReLU	87.78%

Tanh	86.94%
Difference	**+0.83% (ReLU)**

The results showed that the **ReLU activation function slightly outperformed Tanh**, achieving **87.78% validation accuracy** compared to **86.94%**. This small advantage (+0.83%) is due to ReLU's efficiency in handling positive values and faster training. However, both functions remain suitable for this task, with **ReLU being the preferred default choice** unless specific requirements favor Tanh.

Q7: Final Observations

- The model achieved a **final test accuracy of 66.71%**, with stable performance after the 8th epoch.
- **Feature maps** demonstrate a hierarchical extraction of features:
    - Early layers detect basic edges.
    - Later layers identify more complex patterns.
- These results align with theoretical principles of CNNs, with potential for improvement through techniques like **increasing model depth** or **Batch Normalization**.